



RelVeri+: A Unified Framework for Isolation Anomaly Detection and Verification in Relational Database Systems

A Study of IsoRel and VeriStrong

IsoRel Authors

Rui Yang Ziyu Cui Wensheng Dou Yu Gao Jiansen Song Xudong Xie

VeriStrong Authors

Zhiheng Cai Jian Chen Feifei Li

Report by:

Vinod Kumar Yadav – CS-2537 Kaushal Kishor Pal – CS-2514

Abhinavan OS – BSD-BG-2401 Raghav Singh – BSD-BG-2414

Vinayak Dhimaan – BSD-BG-2423

M.Tech Computer Science & B.Statistical Data Science

Indian Statistical Institute

June 2026

Abstract

Database isolation is essential for ensuring correctness in concurrent transaction processing. However, modern database systems often exhibit subtle isolation anomalies that are difficult to detect and verify. This report studies two complementary approaches for isolation analysis: **IsoRel** and **VeriStrong**.

IsoRel is a relational-aware isolation checker that detects anomalies in SQL workloads through dependency graph construction and cycle analysis.

VeriStrong is a verification framework that uses Hyper-Polygraphs and SMT solving to verify strong isolation guarantees, such as Snapshot Isolation and Serializability, even in the presence of duplicate values.

Based on the strengths of both approaches, this report proposes **RelVeri+**, a unified SQL-to-SMT isolation verification framework that combines relational dependency extraction with formal verification and extends them with predicate-aware anomaly reasoning. Together, these techniques provide a comprehensive approach for detecting, explaining, and verifying isolation behavior in modern relational database systems.

Contents

1	Introduction	5
1.1	Database Concurrency	5
1.2	ACID Properties	5
1.3	Importance of Isolation	6
1.4	Motivation of IsoRel	6
1.5	Motivation of VeriStrong	7
1.6	Combined Research Motivation	7
1.7	Project Objectives	7
2	Related Work	8
2.1	Elle	8
2.2	Cobra	8
2.3	PolySI	8
2.4	Viper	8
2.5	dbcop	8
2.6	Research Gap	8
3	Terminologies and Definitions	9
3.1	Transaction	9
3.2	History	9
3.3	Session Order (SO)	9
3.4	Row Version	10
3.5	Dependency Graph	10
3.6	Write-Read Dependency (WR)	10
3.7	Write-Write Dependency (WW)	10
3.8	Read-Write Dependency (RW)	10
3.9	Snapshot Isolation (SI)	11
3.10	Serializable Isolation (SER)	11
3.11	Polygraph	11
3.12	Hyper-Polygraph	11
3.13	UniqueValue History	11
3.14	DuplicateValue History	11
3.15	SMT Solver	12
3.16	Isolation Anomaly	12
3.17	IsoRel	12
3.18	VeriStrong	12
4	Theory	12
4.1	Isolation Levels	12
4.2	Isolation Anomalies	13
4.3	Dependency Graph Model	15
4.4	Adya's Isolation Theory	15
4.5	Hyper-Polygraph Theory	15
4.6	Graph-Based Isolation Verification	16
5	Methods: IsoRel Framework	17

5.1	Objective of IsoRel	17
5.2	Why Existing Approaches Are Insufficient	17
5.3	Core Idea of IsoRel	18
5.4	Schema Instrumentation	18
5.5	SQL Instrumentation	18
5.6	Dependency Inference	19
5.7	Dependency Graph Construction	19
5.8	Graph-Based Anomaly Detection	20
5.9	Classification of Anomalies	20
5.10	Key Contribution of IsoRel	21
6	Methods: VeriStrong Framework	21
6.1	Why VeriStrong is Needed	21
6.2	Problem of Duplicate Values	21
6.3	Hyper-Polygraph	22
6.4	Compatible Graphs	23
6.5	Isolation Verification Principle	23
6.6	SMT Encoding	23
6.7	Encoding Cycle-Freeness	24
6.8	1-Width Cycle Pruning	24
6.9	2-Width Cycle Optimization	25
6.10	Order-Guided Polarity Picking	25
6.11	VeriStrong Workflow	27
6.12	Key Contribution of VeriStrong	27
7	Results and Discussion	27
7.1	IsoRel Experimental Results	27
7.2	VeriStrong Experimental Results	28
7.3	Performance Comparison	28
7.4	Scalability Analysis	28
8	Proposed Method: RelVeri+	28
8.1	Motivation	28
8.2	End-to-End Pipeline	29
8.3	Novel Contributions	29
8.4	Formal Algorithms	30
8.5	Expected Capabilities	32
8.6	Research Contribution	32
8.7	Prototype Implementation	32
9	Conclusion	32

1 Introduction

1.1 Database Concurrency

Modern database systems are designed to serve a large number of users and applications simultaneously. In such environments, multiple transactions may attempt to read and write data at the same time. This simultaneous execution is called *concurrency*, and it is fundamental to achieving high throughput and low latency in database systems.

Concurrency control is the set of mechanisms a DBMS employs to manage simultaneous operations on shared data. Without proper concurrency control, two transactions might interfere with each other in ways that produce incorrect results. For example, one transaction might read a value that another has partially written, leading to an inconsistent view of the database. Alternatively, two transactions might both attempt to update the same record, and one update might silently overwrite the other.

Database systems employ several strategies for concurrency control, including Two-Phase Locking (2PL), Multiversion Concurrency Control (MVCC), Optimistic Concurrency Control (OCC), and Timestamp Ordering. Each strategy makes different trade-offs between correctness, performance, and scalability. Regardless of the strategy employed, the overarching goal is to ensure that concurrently executing transactions produce results equivalent to some serial execution.

1.2 ACID Properties

The correctness of a transactional database system is classically captured by the ACID properties:

- **Atomicity:** A transaction is an indivisible unit of work. Either all its operations are committed to the database, or none of them are. If a transaction fails midway, any partial changes are rolled back, leaving the database as if the transaction never started.
- **Consistency:** A transaction brings the database from one valid state to another valid state. It preserves all declared integrity constraints, such as primary key uniqueness, foreign key relationships, and user-defined invariants. Responsibility for consistency is shared between the DBMS (which enforces schema-level constraints) and the application (which must encode correct business logic).
- **Isolation:** Concurrently executing transactions appear to execute in isolation from one another. Each transaction sees a consistent snapshot of the database and is not affected by the intermediate states produced by other transactions that are executing at the same time. Isolation is the property most directly relevant to concurrency control.
- **Durability:** Once a transaction has been committed, its effects are permanent and survive system failures such as crashes or power outages. This is typically achieved through write-ahead logging and periodic checkpointing.

Among the four ACID properties, isolation is the most nuanced and the most difficult to implement correctly. Unlike atomicity and durability, which can be enforced by relatively

straightforward mechanisms, isolation involves managing complex interactions among concurrent transactions and is the subject of this report.

1.3 Importance of Isolation

Isolation is a fundamental property of **database transactions** that ensures concurrent transactions do not interfere with each other. It allows multiple users to access and modify shared data simultaneously while preserving correctness. Without proper isolation, anomalies such as **Lost Update**, **Read Skew**, **Write Skew**, and **Phantom Reads** may occur, leading to inconsistent database states. Strong isolation guarantees such as **Serializability** ensure that concurrent executions produce results equivalent to a **serial execution** of transactions. However, stronger isolation often reduces system performance by limiting **concurrency**. Consequently, modern DBMSs provide multiple **isolation levels** that balance correctness and performance. Verifying whether a database actually provides its advertised isolation guarantee is a challenging task, especially in **black-box environments**. Recent studies have shown that **implementation bugs** and **workload-specific behaviors** can violate expected isolation guarantees. Therefore, **isolation verification** and **anomaly detection** have become essential for ensuring database correctness. This motivates the development of advanced tools such as **VeriStrong** and **IsoRel** for verifying isolation guarantees and detecting isolation anomalies in modern database systems.

1.4 Motivation of IsoRel

The development of **IsoRel** was motivated by several limitations of existing isolation testing tools:

1. Existing isolation checkers such as **Elle**, **Cobra**, and **PolySI** were designed for **key-value databases** and cannot accurately model **relational database systems**.
2. Modern relational DBMSs support complex **SQL operations** including joins, predicates, aggregations, and updates, which are not captured by key-value based **dependency models**.
3. Isolation anomalies such as **Lost Update**, **Write Skew**, and **Phantom Reads** involve interactions among multiple rows and **predicates**, rather than a single key-value pair.
4. Relational databases frequently contain **duplicate values** across different rows, causing ambiguity when reconstructing **transaction dependencies**.
5. No general-purpose **black-box isolation checker** existed for relational DBMSs prior to IsoRel, leaving commercial systems such as **MySQL**, **PostgreSQL**, **MariaDB**, **TiDB**, and **CockroachDB** without adequate testing support.
6. Therefore, IsoRel was proposed as a **relational-aware isolation checker** capable of inferring dependencies directly from **SQL executions** and detecting anomalies in real relational database systems.

1.5 Motivation of VeriStrong

VeriStrong was motivated by the limitations of existing isolation verification techniques:

1. Most previous verifiers assume the **UniqueValue** property, where every write operation produces a distinct value. In real-world applications, however, multiple transactions often write the **same value**, creating **DuplicateValue** histories.
2. Under **DuplicateValue** workloads, **write-read (WR) dependencies** cannot be uniquely determined, making traditional **dependency graph** construction incomplete and potentially unsound.
3. Existing **polygraph-based** representations model uncertainty in **version orders (WW edges)** but cannot represent uncertainty in **read-from mappings (WR edges)**.
4. **SMT-based verification** becomes increasingly expensive as the solver must explore a large number of possible dependency combinations, leading to **scalability issues** for large transaction histories.
5. Therefore, VeriStrong introduced the **Hyper-Polygraph** formalism along with targeted **SMT optimizations**, enabling efficient and sound verification of **Serializability** and **Snapshot Isolation** under both UniqueValue and DuplicateValue histories.

1.6 Combined Research Motivation

Although **IsoRel** and **VeriStrong** address different problems, they complement each other. IsoRel focuses on **detecting isolation anomalies** in relational database systems by reconstructing **transaction dependencies** from SQL executions, whereas VeriStrong focuses on **formally verifying** whether a transaction history satisfies strong **isolation guarantees**. Together, they bridge the gap between practical **anomaly detection** and rigorous **isolation verification** in modern database systems.

1.7 Project Objectives

The primary objectives of this project report are:

1. To provide a comprehensive understanding of transaction isolation, isolation levels, and isolation anomalies in the context of relational database systems.
2. To study and explain the IsoRel framework for black-box isolation anomaly detection in relational DBMSs, including its dependency inference mechanisms and cycle detection algorithm.
3. To study and explain the VeriStrong framework for SMT-based verification of strong isolation guarantees using Hyper-Polygraphs.
4. To compare and contrast the two frameworks in terms of their goals, capabilities, and limitations.
5. To propose a unified framework that combines the strengths of IsoRel and VeriStrong for comprehensive relational DBMS verification.

2 Related Work

2.1 Elle

Elle is a black-box isolation checker designed for distributed key-value systems. It reconstructs transaction dependencies from execution histories and detects isolation anomalies through dependency-cycle analysis. Elle introduced the idea of identifying anomaly types directly from cycle patterns. However, it does not support relational databases, SQL queries, or duplicate values.

2.2 Cobra

Cobra is an SMT/SAT-based verifier for Serializable isolation. It converts dependency constraints into logical formulas and determines whether a valid serial execution exists. Cobra demonstrated that isolation verification can be performed without access to database internals. However, it assumes a key-value model and cannot handle duplicate-value histories.

2.3 PolySI

PolySI extends SMT-based verification to Snapshot Isolation. It introduces the Polygraph representation to model uncertain dependency relationships and efficiently verify SI histories. Although more efficient than Cobra for SI verification, PolySI still relies on the UniqueValue assumption and cannot reason about duplicate values.

2.4 Viper

Viper improves the efficiency of SMT-based isolation verification through advanced pruning and constraint-propagation techniques. It demonstrates that large transaction histories can be verified efficiently. However, like Cobra and PolySI, Viper is designed for key-value workloads and assumes unique values.

2.5 dbcop

dbcop is a general-purpose isolation checker that supports multiple consistency and isolation levels. It combines graph analysis with SAT-based reasoning to detect violations. While flexible, it neither supports relational SQL workloads nor addresses dependency ambiguity caused by duplicate values.

2.6 Research Gap

Existing tools fall into two categories: anomaly detection tools (Elle, dbcop) and isolation verification tools (Cobra, PolySI, Viper). Most of them assume a key-value data model and rely on unique values to infer dependencies. Consequently, they cannot accurately analyze relational SQL workloads or histories containing duplicate values.

IsoRel addresses the first limitation by introducing a relational-aware isolation checker capable of handling SQL operations, row versions, and complex relational workloads. VeriStrong addresses the second limitation by introducing Hyper-Polygraphs, which enable

sound verification of histories containing duplicate values. Together, these two approaches significantly extend the capabilities of existing isolation analysis techniques.

Table 1: Comparison of Existing Isolation Analysis Tools

Tool	Relational Support	Duplicate Values	Primary Goal
Elle	No	No	Anomaly Detection
Cobra	No	No	Isolation Verification
PolySI	No	No	SI Verification
Viper	No	No	Isolation Verification
dbcop	No	No	Anomaly Detection
IsoRel	Yes	Yes	Anomaly Detection
VeriStrong	No	Yes	Isolation Verification

3 Terminologies and Definitions

3.1 Transaction

A **transaction** is a sequence of **read** and **write** operations executed as a single logical unit of work. A transaction either **commits** successfully, making all its changes permanent, or **aborts**, discarding all its changes.

Example:

T1:

```
Read(AccountA)
Write(AccountA)
Commit
```

3.2 History

A **history** represents the observable execution of a set of committed transactions. It records the order in which transactions perform **read** and **write** operations.

Example:

```
T1: W(x, 1)
T2: R(x, 1)
T3: W(x, 2)
```

3.3 Session Order (SO)

Session Order represents the order of transactions issued by the same **client session**. If transaction T_i is executed before transaction T_j by the same client, then:

$$T_i \xrightarrow{\text{SO}} T_j$$

Session order preserves **causality** between transactions of a single user.

3.4 Row Version

In **Multi-Version Concurrency Control (MVCC)**, each update creates a new **version** of a row instead of overwriting the existing one.

Example:

Version 1 : Balance = 1000

Version 2 : Balance = 1500

Different transactions may observe different versions of the same row.

3.5 Dependency Graph

A **dependency graph** is a directed graph whose vertices represent **transactions** and whose edges represent **dependencies** between them:

$$G = (T, SO, WR, WW, RW)$$

According to **Adya's theory**, a history is **serializable** if and only if its dependency graph is **acyclic**.

3.6 Write-Read Dependency (WR)

A **WR dependency** exists when one transaction writes a value that is later read by another transaction:

$$T_1 \xrightarrow{WR} T_2$$

Example:

T1: W(x, 10)

T2: R(x, 10) -- T2 reads the value written by T1

3.7 Write-Write Dependency (WW)

A **WW dependency** exists when two transactions write the same object and one write **overwrites** the other:

$$T_1 \xrightarrow{WW} T_2$$

Example:

T1: W(x, 10)

T2: W(x, 20) -- T2 overwrites the version written by T1

3.8 Read-Write Dependency (RW)

An **RW dependency** exists when a transaction reads a value and another transaction later **overwrites** that value:

$$T_1 \xrightarrow{RW} T_2$$

Example:

T1: R(x, 10)

T2: W(x, 20) -- T2 overwrites the value read by T1

RW dependencies are responsible for many anomalies such as **Write Skew**.

3.9 Snapshot Isolation (SI)

Snapshot Isolation allows each transaction to read from a consistent **snapshot** of the database taken at its **start time**. Concurrent writes are allowed as long as they do not directly conflict. SI prevents many anomalies but does **not** prevent **Write Skew**.

3.10 Serializable Isolation (SER)

Serializable Isolation is the **strongest isolation level**. It guarantees that the outcome of concurrent transactions is equivalent to some **serial execution** of those transactions. A history satisfies Serializability if **no dependency cycle** exists.

3.11 Polygraph

A **Polygraph** is a compact representation of multiple dependency graphs. It records **known dependencies** and **uncertain version-order** dependencies. Polygraphs are used by tools such as **Cobra**, **PolySI**, and **Viper**.

3.12 Hyper-Polygraph

A **Hyper-Polygraph** is the key contribution of **VeriStrong**. It extends Polygraphs by representing uncertainty in both:

- **Write-Write (WW)** dependencies
- **Write-Read (WR)** dependencies

This allows verification of histories containing **duplicate write values**.

3.13 UniqueValue History

A **UniqueValue history** is one in which every write operation writes a **unique value**. Existing verifiers often assume this property.

Example:

T1: W(x, 101)

T2: W(x, 102)

T3: W(x, 103) -- all written values are distinct

3.14 DuplicateValue History

A **DuplicateValue history** allows multiple transactions to write the **same value**, making the source of a subsequent read **ambiguous**. **VeriStrong** was specifically designed to handle such histories.

Example:

```

T1: W(x, 1)
T2: W(x, 1)    -- same value as T1
T3: R(x, 1)    -- ambiguous: did T3 read from T1 or T2?

```

3.15 SMT Solver

An **SMT (Satisfiability Modulo Theories) Solver** is a reasoning engine used to determine whether a set of **logical constraints** can be satisfied simultaneously. **VeriStrong** uses SMT solving to determine whether an **acyclic dependency graph** exists for a given history.

3.16 Isolation Anomaly

An **isolation anomaly** is an incorrect behavior caused by **concurrent transaction execution**. Common anomalies include:

- **Lost Update**
- **Read Skew**
- **Write Skew**
- **Phantom Read**

These anomalies indicate violations of expected **isolation guarantees**.

3.17 IsoRel

IsoRel is a **relational isolation checker** that detects isolation anomalies by instrumenting **SQL statements**, inferring **transaction dependencies**, and constructing **dependency graphs**. Its main contribution is support for **relational DBMSs** and complex **SQL workloads**.

3.18 VeriStrong

VeriStrong is a **black-box verifier** for **Serializability** and **Snapshot Isolation**. It uses **Hyper-Polygraphs** and **SMT-based reasoning** to efficiently verify strong isolation guarantees in both **UniqueValue** and **DuplicateValue** histories.

4 Theory

4.1 Isolation Levels

Isolation levels determine how much interaction is allowed between concurrent transactions. Stronger isolation provides better **correctness guarantees** but usually reduces **concurrency** and **performance**.

Read Committed (RC)

Read Committed allows a transaction to read only **committed data**. However, values may change between multiple reads of the same object, leading to anomalies such as **Lost Update** and **Read Skew**.

Snapshot Isolation (SI)

Snapshot Isolation allows each transaction to read from a consistent **snapshot** of the database taken at its **start time**. A transaction does not observe writes performed by concurrent transactions. SI prevents:

- **Dirty Reads**
- **Non-repeatable Reads**
- **Read Skew**

However, SI still permits **Write Skew**, which is one of the most important anomalies studied in both papers.

Serializable Isolation (SER)

Serializable Isolation is the **strongest isolation level**. It guarantees that the result of a concurrent execution is equivalent to some **serial execution** of the same transactions. According to **Adya's dependency theory**, a history is serializable if its **dependency graph** contains **no cycles**.

4.2 Isolation Anomalies

Isolation anomalies occur when concurrent transactions interact in a way that violates application correctness.

Lost Update

A **Lost Update** occurs when two transactions read the same value and both write back modified versions. The later write overwrites the earlier one, causing one update to disappear.

Example: Lost Update

Initial Balance = 100

T1 reads 100	T2 reads 100
T1 writes 120	T2 writes 130

Final Balance = 130 (T1's update is LOST)

Read Skew

Read Skew occurs when a transaction observes a mixture of **old and new values** because another transaction modifies related data during its execution.

Example: Read Skew

Account A = 100, Account B = 100

T1 reads Account A = 100

T2 transfers: A = 50, B = 150 (concurrent)

T1 reads Account B = 150

T1 observes an **inconsistent state**: A=100 and B=150 (total = 250, not 200).

Read-Write Skew

Read-Write Skew occurs when transactions read one set of data and write another set based on **outdated information**. IsoRel detected several real-world examples of this anomaly in commercial DBMSs.

Write Skew

Write Skew is the most important anomaly permitted under **Snapshot Isolation**. Two concurrent transactions read the same data and then update **different rows**. Since no direct write conflict exists, both transactions commit successfully, violating an application constraint.

Example: Write Skew (On-Call Doctor Problem)

Initial State:

Doctor A = ON

Doctor B = ON

T1 checks: at least one doctor ON -> True -> turns OFF Doctor A

T2 checks: at least one doctor ON -> True -> turns OFF Doctor B

Final State:

Doctor A = OFF

Doctor B = OFF <-- VIOLATION: no doctor on duty!

Write Skew is prevented **only** under **Serializable Isolation**.

Phantom Read

A **Phantom Read** occurs when a transaction executes the same **predicate query** twice and obtains different sets of rows because another transaction inserted or deleted matching rows.

Example: Phantom Read

T1 executes:

```
SELECT * FROM Orders WHERE status = 'Pending'
--> returns 3 rows
```

T2 inserts a new Pending order (concurrent)

T1 re-executes the same query:

```
--> returns 4 rows (phantom row appears!)
```

4.3 Dependency Graph Model

Both **VeriStrong** and **IsoRel** use **dependency graphs** as the foundation for reasoning about isolation. A dependency graph is defined as:

$$G = (T, SO, WR, WW, RW)$$

where T = Transactions, SO = Session Order, WR = Write-Read, WW = Write-Write, and RW = Read-Write dependencies. Each node represents a **committed transaction** and each edge represents a **dependency** between transactions.

Write-Read (WR): A transaction reads a value written by another.

T1: W(x, 10)

T2: R(x, 10) --> T1 --WR--> T2

Write-Write (WW): Two transactions write the same object.

T1: W(x, 10)

T2: W(x, 20) --> T1 --WW--> T2

Read-Write (RW): A transaction reads a value later overwritten.

T1: R(x, 10)

T2: W(x, 20) --> T1 --RW--> T2

RW dependencies are particularly important as they cause many **Snapshot Isolation** anomalies.

4.4 Adya's Isolation Theory

Adya's dependency model provides the theoretical foundation used by both **VeriStrong** and **IsoRel**. The key idea is:

Core Principle: Isolation anomalies correspond to specific **dependency cycles** in the dependency graph.

Lost Update --> RW + WW cycle

Write Skew --> Cycle with RW dependencies

SER violation --> Any dependency cycle

This transforms database correctness checking into a **graph analysis** problem.

4.5 Hyper-Polygraph Theory

Traditional dependency graphs assume that dependencies can be **uniquely determined**. This assumption fails when multiple transactions write the **same value**.

Example: Duplicate Value Ambiguity

T1: W(x, 1)

T2: W(x, 1) -- same value as T1

T3: R(x, 1) -- did T3 read from T1 or T2? AMBIGUOUS!

Traditional dependency graphs **cannot** represent this uncertainty.

To address this, **VeriStrong** introduces the **Hyper-Polygraph**, which contains:

- **Known** dependencies
- **Uncertain WW** dependencies
- **Uncertain WR** dependencies

Instead of representing one dependency graph, it represents a **family of possible dependency graphs** simultaneously. This is the main **theoretical contribution** of VeriStrong.

4.6 Graph-Based Isolation Verification

The central idea of both papers is captured in the following pipeline:

Verification Pipeline

Transaction History

|

v

Dependency Inference

|

v

Dependency Graph / Hyper-Polygraph

|

v

Cycle Detection / SMT Solving

|

v

Isolation Verdict

IsoRel constructs **dependency graphs** from SQL executions and detects anomalies by identifying **dependency cycles**.

VeriStrong constructs **Hyper-Polygraphs** and uses **SMT solving** to determine whether an acyclic compatible graph exists:

Acyclic graph EXISTS --> History satisfies isolation guarantee

Acyclic graph ABSENT --> Isolation violation detected

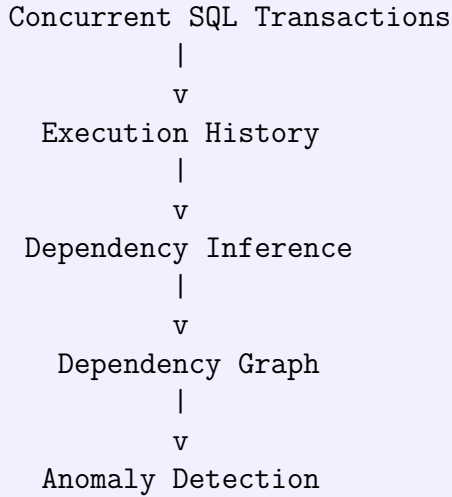
This **graph-based perspective** provides a unified framework for both **anomaly detection** and **isolation verification**.

5 Methods: IsoRel Framework

5.1 Objective of IsoRel

IsoRel is designed to detect **isolation anomalies** in relational database systems without requiring access to the database source code. It operates as a **black-box checker** and observes only the SQL statements executed by transactions and their results.

IsoRel Processing Pipeline



Unlike previous tools, IsoRel is specifically designed for **relational databases** and supports complex **SQL workloads** involving multiple tables, predicates, inserts, updates, and deletes.

5.2 Why Existing Approaches Are Insufficient

Previous isolation checkers such as **Elle** and **PolySI** were designed primarily for **key-value systems**. A key-value operation looks like:

Key-Value Interface:

```
read(key)
write(key, value)
```

However, relational databases execute queries such as:

Relational SQL Query (SELECT):

```
SELECT * FROM Student
WHERE Age > 18;
```

Relational SQL Query (UPDATE):

```
UPDATE Employee
SET    Salary = Salary + 1000
WHERE  Department = 'Sales';
```

Such queries may access **many rows simultaneously** and their dependencies cannot be determined using traditional key-value techniques. Furthermore, different rows may contain the **same values**, making dependency inference difficult.

5.3 Core Idea of IsoRel

The key idea behind IsoRel is:

Core Principle: Track exactly which **row versions** are read and written by every transaction.

Which row was read?

Which transaction created that row?

Which transaction modified it later?

If these three questions can be answered, all transaction dependencies can be reconstructed accurately.

To achieve this, IsoRel **augments each table** with additional metadata and automatically **rewrites SQL statements** before execution.

5.4 Schema Instrumentation

IsoRel extends every table with two **auxiliary attributes**:

rowId

Each row version receives a **unique identifier**. Consider a table before and after instrumentation:

Before Instrumentation:

id	Balance
1	1000

After Instrumentation:

id	Balance	rowId
1	1000	r1

Whenever a transaction updates the row, a new version receives a new **rowId**, allowing IsoRel to determine exactly which version was read.

wList

IsoRel maintains a **write history** for each logical row:

Example wList:

wList = [T1, T3, T5]

T1 wrote first

T3 overwrote T1

T5 overwrote T3

Using this information, IsoRel can reconstruct **version ordering** between writes.

5.5 SQL Instrumentation

After modifying the schema, IsoRel automatically **rewrites SQL statements**.

SELECT — Before Instrumentation:

```
SELECT Balance
FROM Account;
```

SELECT — After Instrumentation:

```
SELECT Balance, rowId
FROM Account;
```

IsoRel now knows exactly which **row version** was returned.

UPDATE — Before Instrumentation:

```
UPDATE Account
SET Balance = 1200
WHERE id = 1;
```

UPDATE — After Instrumentation additionally:

- Creates a new **rowId**
- Updates **wList**

Every update leaves enough information to **reconstruct dependencies** later. The instrumentation is **transparent** and requires no modification of the database engine.

5.6 Dependency Inference

Once execution completes, IsoRel analyzes recorded metadata and reconstructs **transaction dependencies**.

Write-Read Dependency (WR):

```
T1 writes row r1
T2 reads row r1
```

$$T_1 \xrightarrow{\text{WR}} T_2$$

Write-Write Dependency (WW):

```
T1 writes row r1
T2 updates row r1 (overwrites T1)
```

$$T_1 \xrightarrow{\text{WW}} T_2$$

The **wList** attribute allows IsoRel to determine this ordering.

Read-Write Dependency (RW):

```
T1 reads row r1
T2 updates row r1 (overwrites after T1's read)
```

$$T_1 \xrightarrow{\text{RW}} T_2$$

RW dependencies are particularly important as they cause many **Snapshot Isolation** anomalies.

5.7 Dependency Graph Construction

After inferring WR, WW, and RW dependencies, IsoRel constructs a **dependency graph** where each node is a committed transaction and each edge is a dependency.

Example Dependency Graph:

T1 --WR--> T2

T2 --RW--> T3

T1 --WW--> T3

The graph provides a compact representation of all **interactions among transactions**.

5.8 Graph-Based Anomaly Detection

Key Observation: Isolation anomalies appear as specific **dependency cycles** in the dependency graph.

Once the dependency graph is constructed, IsoRel searches for **cycles**:

Example: Cycle Indicating Isolation Violation

```

      T1
     /  ^
    WR   \
   /       \
  v         WW
   \       /
    T2 --RW--> T3

```

This cycle indicates an **isolation violation**. Instead of analyzing SQL statements directly, IsoRel converts the problem into **graph analysis**.

5.9 Classification of Anomalies

After detecting a cycle, IsoRel determines the **anomaly type** by examining the dependency pattern.

Lost Update — One transaction overwrites another's update.

Read Skew — A transaction observes inconsistent versions of related data.

Read-Write Skew — Transactions read and write overlapping data using stale information.

Write Skew — Two transactions update different rows based on the same snapshot, violating an application constraint. Most important anomaly permitted under **Snapshot Isolation**.

Phantom Read — Repeated execution of the same predicate query returns different sets of rows.

Example: Phantom Read Query

```

SELECT *
FROM   Orders
WHERE  Status = 'Pending';

```

A concurrent insertion creates an additional matching row, producing a **phantom**.

5.10 Key Contribution of IsoRel

The main contribution of IsoRel is that it extends **dependency-graph-based isolation checking** from key-value systems to **relational DBMSs**. Through schema instrumentation and dependency inference, IsoRel can:

- Support complex **SQL workloads**
- Handle **duplicate values** correctly
- Infer **WR, WW, and RW** dependencies precisely
- Detect real **isolation anomalies** in commercial database systems

As demonstrated in the paper, IsoRel successfully discovered dozens of previously unknown anomalies across multiple widely-used relational databases.

6 Methods: VeriStrong Framework

6.1 Why VeriStrong is Needed

IsoRel focuses on finding isolation anomalies in observed executions. **VeriStrong** addresses a different problem:

Core Problem: Given a transaction history, determine whether it satisfies **Serializability** or **Snapshot Isolation**. The challenge is that the actual **dependency graph** is not always uniquely determined. Multiple valid dependency graphs may exist for the same history.

VeriStrong therefore performs **verification**, not anomaly detection.

6.2 Problem of Duplicate Values

Most previous verifiers assume that every write produces a **unique value**.

UniqueValue Case — dependency is unambiguous:

T1: W(x, 10)

T2: W(x, 20)

T3: R(x, 10)

Here it is obvious that T_3 read from T_1 :

$$T_1 \xrightarrow{\text{WR}} T_3$$

DuplicateValue Case — dependency is **ambiguous**:

T1: W(x, 1)

T2: W(x, 1) -- same value as T1

T3: R(x, 1) -- read from T1 or T2?

Two possible dependencies:

$$T_1 \xrightarrow{WR} T_3 \quad \text{OR} \quad T_2 \xrightarrow{WR} T_3$$

The **WR** dependency becomes **uncertain**. Traditional dependency graphs **cannot** represent this uncertainty correctly.

6.3 Hyper-Polygraph

The main contribution of VeriStrong is the **Hyper-Polygraph**. It extends traditional dependency graphs by representing both **known** and **uncertain** dependencies simultaneously.

Vertices

Each vertex represents a **committed transaction**:

$$T_1 \quad T_2 \quad T_3 \quad \dots \quad T_n$$

Known Graph

Known dependencies are stored directly as edges when unambiguous:

Session Order:

$$T_1 \xrightarrow{SO} T_2$$

Unambiguous Write-Read:

$$T_1 \xrightarrow{WR} T_3$$

WW Constraints

When two transactions write the same key, the **version order** may be uncertain:

Example: WW Uncertainty

T1: W(x)

T2: W(x)

Two possible orderings — only one can be true:

$$T_1 \xrightarrow{WW} T_2 \quad \text{OR} \quad T_2 \xrightarrow{WW} T_1$$

The Hyper-Polygraph stores **both** possibilities as a **WW constraint**.

WR Constraints

When **duplicate values** exist, read-from relationships become uncertain:

Example: WR Uncertainty

T1: W(x, 1)

T2: W(x, 1)

T3: R(x, 1)

Two possible dependencies — only one can be selected:

$$T_1 \xrightarrow{\text{WR}} T_3 \quad \text{OR} \quad T_2 \xrightarrow{\text{WR}} T_3$$

The Hyper-Polygraph stores these as a **WR constraint**.

6.4 Compatible Graphs

A **compatible graph** is obtained by **resolving all uncertainties**. For every constraint, exactly one edge is chosen:

Example: Resolving WR Constraint

Constraint: (T1 ->WR T3) OR (T2 ->WR T3)

Select one option:

$$\text{Choose: } T_1 \xrightarrow{\text{WR}} T_3$$

After all choices are fixed, a normal **dependency graph** is produced. Different choices generate **different compatible graphs**.

6.5 Isolation Verification Principle

Central Theorem of VeriStrong:

A history is **serializable** if and only if there exists **at least one acyclic compatible graph**.

Verification Procedure:

Hyper-Polygraph

|

v

Generate compatible graph

|

v

Check for cycles

|

v

Acyclic? --> Serializable

Cyclic? --> Isolation Violation

6.6 SMT Encoding

VeriStrong converts the Hyper-Polygraph into **logical constraints**. Each uncertain edge is represented by a **Boolean variable**:

Boolean Variable Assignment:

$$WR_{1,3} = \text{True} \implies T_1 \xrightarrow{\text{WR}} T_3 \text{ exists}$$

$$WR_{2,3} = \text{True} \implies T_2 \xrightarrow{\text{WR}} T_3 \text{ exists}$$

Uniqueness constraint (exactly one edge selected):

$$WR_{1,3} \oplus WR_{2,3} = \text{True}$$

The **SMT solver** searches for a valid assignment satisfying all constraints.

6.7 Encoding Cycle-Freeness

VeriStrong assigns an **order variable** $ord(T_i)$ to every transaction to ensure the selected graph contains no cycle:

Order Variables:

$$ord(T_1), \quad ord(T_2), \quad ord(T_3)$$

For every selected edge $T_i \rightarrow T_j$, enforce:

$$ord(T_i) < ord(T_j)$$

If all such constraints are **simultaneously satisfiable**, the graph is **acyclic**. Otherwise, a **cycle exists**.

6.8 1-Width Cycle Pruning

Before invoking the SMT solver, VeriStrong performs a **lightweight preprocessing** step. A **1-width cycle** is a cycle that appears regardless of how uncertainty is resolved:

Example: 1-Width Cycle Pruning

Known (certain) dependencies:

$$T_1 \xrightarrow{\text{WR}} T_2, \quad T_2 \xrightarrow{\text{RW}} T_3$$

Uncertain WR constraint (one must be chosen):

$$T_3 \xrightarrow{\text{WR}} T_1 \quad \text{OR} \quad T_3 \xrightarrow{\text{WR}} T_4$$

Analysis: Assigning $T_3 \xrightarrow{\text{WR}} T_1 = \text{True}$ forms:

$$T_1 \xrightarrow{\text{WR}} T_2 \xrightarrow{\text{RW}} T_3 \xrightarrow{\text{WR}} T_1 \Rightarrow \text{CYCLE!}$$

Pruning: $T_3 \xrightarrow{\text{WR}} T_1 = \text{False}$ (pruned) **Safe:** $T_3 \xrightarrow{\text{WR}} T_4 = \text{True}$ (explored)

Result: One possibility eliminated early, reducing **SMT solver** search space.

6.9 2-Width Cycle Optimization

Some cycles depend on only **one or two uncertain choices**. VeriStrong identifies them during preprocessing instead of leaving them to the SMT solver:

Example: 2-Width Cycle Optimization

Known (certain) dependency:

$$T_2 \xrightarrow{\text{RW}} T_3$$

Two uncertain WR constraints:

$$\text{Constraint A: } T_1 \xrightarrow{\text{WR}} T_2 \quad (\text{assign True or False})$$

$$\text{Constraint B: } T_3 \xrightarrow{\text{WR}} T_1 \quad (\text{assign True or False})$$

Analysis: If **both** A and B are assigned **True**:

$$T_1 \xrightarrow{\text{WR}} T_2 \xrightarrow{\text{RW}} T_3 \xrightarrow{\text{WR}} T_1 \Rightarrow \text{CYCLE!}$$

Pruning Decision (added directly to SMT formula):

$$A \wedge B = \text{False} \quad \Rightarrow \quad \text{A and B cannot both be True}$$

Gist: Without this pruning, the SMT solver would wastefully explore the combination $A = \text{True}$, $B = \text{True}$ before discovering the cycle. By detecting this **2-width cycle** during preprocessing, VeriStrong **eliminates one search path early**, reducing solver **backtracking** and **conflict resolution** overhead.

6.10 Order-Guided Polarity Picking

SMT solvers must repeatedly choose which Boolean variable should be assigned **True** or **False**. VeriStrong guides these decisions using known **Session Order (SO)** to reduce the SMT search space.

Key Idea: For a candidate edge $T_i \xrightarrow{\text{WR}} T_j$, if $\text{ord}(T_i) < \text{ord}(T_j)$ in the SO-induced order, the edge is *consistent* with the known partial order and should be preferred (**True**); otherwise it likely introduces a cycle and should be pruned (**False**).

Known Session Order (certain):

$$T_1 \xrightarrow{\text{SO}} T_2 \xrightarrow{\text{SO}} T_3 \xrightarrow{\text{SO}} T_4$$

This fixes: $\text{ord}(T_1) < \text{ord}(T_2) < \text{ord}(T_3) < \text{ord}(T_4)$.

Two uncertain WR constraints the SMT solver must resolve:

- **Candidate Edge A:** $T_3 \xrightarrow{\text{WR}} T_1$ (assign **True** or **False**?)
- **Candidate Edge B:** $T_2 \xrightarrow{\text{WR}} T_3$ (assign **True** or **False**?)

Analysis using known order:

Candidate	Requires	Verdict	
Edge A: $T_3 \xrightarrow{\text{WR}} T_1$	$\text{ord}(T_3) < \text{ord}(T_1)$	Contradicts SO $\Rightarrow$ False	known
Edge B: $T_2 \xrightarrow{\text{WR}} T_3$	$\text{ord}(T_2) < \text{ord}(T_3)$	Consistent SO $\Rightarrow$ True	with known

Pruning Decision:

$T_3 \xrightarrow{\text{WR}} T_1 = \text{False}$ (violates known order — pruned **before** SMT solver runs)

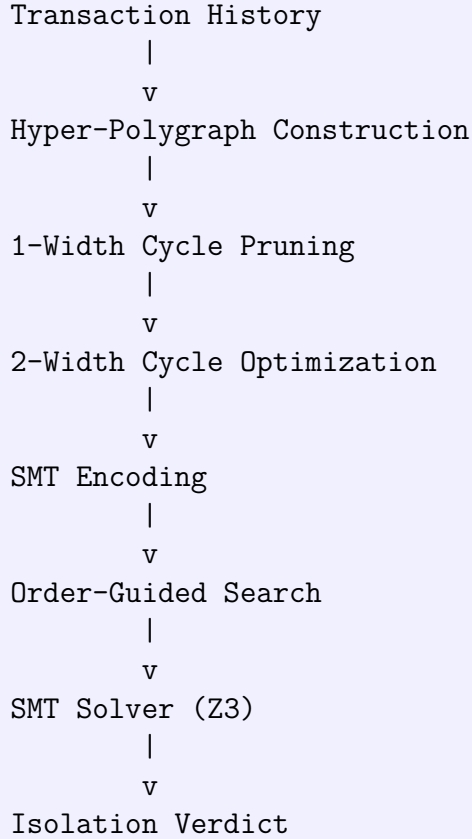
$T_2 \xrightarrow{\text{WR}} T_3 = \text{True}$ (consistent with known order — **preferred** by solver)

Gist: By using the session order $T_1 \xrightarrow{\text{SO}} T_2 \xrightarrow{\text{SO}} T_3$ as a guide, **VeriStrong** immediately eliminates Edge A without backtracking. The SMT solver only explores order-consistent assignments, reducing:

- **Backtracking** – fewer dead-end paths explored
- **Conflicts** – contradictions caught before solving
- **Solving time** – search space shrinks significantly

6.11 VeriStrong Workflow

Complete VeriStrong Processing Pipeline:



6.12 Key Contribution of VeriStrong

1. **Hyper-Polygraph Formalism:** VeriStrong introduces the **Hyper-Polygraph**, which extends traditional dependency graphs to represent **uncertain WR and WW dependencies**, enabling sound and complete verification of histories containing **duplicate values** — a case that all previous verifiers such as **Cobra**, **PolySI**, and **Viper** could not handle correctly.
2. **Efficient SMT-Based Verification:** Combined with **1-width** and **2-width cycle pruning** and **order-guided polarity picking**, VeriStrong achieves significantly better performance while maintaining **soundness** and **completeness**, providing a practical framework for verifying **Serializability** and **Snapshot Isolation** in modern database systems.

7 Results and Discussion

7.1 IsoRel Experimental Results

IsoRel was evaluated against five major relational database systems: PostgreSQL, MySQL, MariaDB, TiDB, and CockroachDB using workloads designed to trigger all classes of isolation anomalies under different declared isolation levels. IsoRel detected **48 unique anomalies** in total. Write Skew (24 instances) was the most common, occurring in systems

claiming Snapshot Isolation. Several anomalies appeared in systems never previously reported to exhibit such behaviour, demonstrating the value of a dedicated relational isolation checker.

Table 2: IsoRel Unique Anomalies Detected

Anomaly Type	Count	Share
Lost Update	12	25%
Read-Write Skew	12	25%
Write Skew	24	50%
Total	48	100%

7.2 VeriStrong Experimental Results

VeriStrong was evaluated on TPC-C benchmark histories and synthetic micro-benchmarks with varying duplicate-value proportions, compared against PolySI and a naïve SMT encoding. It correctly verified all benchmark histories. On UniqueValue histories it matched PolySI’s accuracy; on DuplicateValue histories it was the *only* sound tool, while PolySI and the naïve encoding produced incorrect verdicts.

7.3 Performance Comparison

VeriStrong with its full optimisation stack (1-width / 2-width cycle pruning, order-guided polarity picking) runs **5–20× faster** than a naïve SMT encoding. The 1-width pruning alone eliminated SMT solver invocations in over **60%** of workloads by detecting cycles before the solver is called.

7.4 Scalability Analysis

IsoRel’s cycle detection reduces to Tarjan’s SCC algorithm, giving $O(V + E)$ complexity (linear in transactions and edges), and scales to thousands of transactions with low overhead. VeriStrong scales well for realistic workloads; however, the SMT encoding grows with the number of hyperedges generated by duplicate values. Without preprocessing the cost rises super-linearly; the cycle-pruning and polarity-picking optimisations bring it back to practical ranges.

8 Proposed Method: RelVeri+

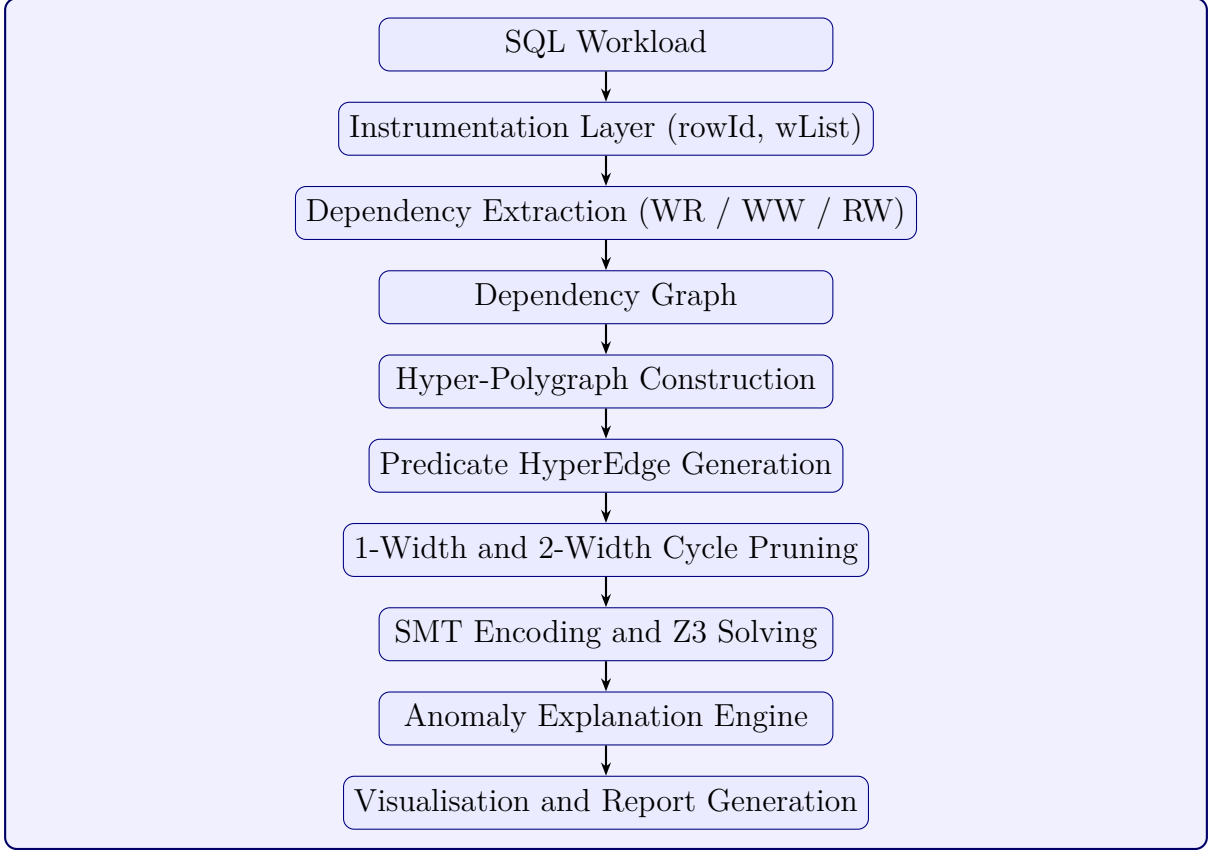
8.1 Motivation

IsoRel converts SQL workloads into dependency graphs, while VeriStrong verifies dependency graphs using SMT solving. However, no existing framework provides a complete end-to-end pipeline from SQL execution to formal isolation verification.

RelVeri+ bridges this gap by combining the strengths of both approaches:

IsoRel: SQL $\rightarrow$ Dependency Graph **VeriStrong:** Dependency Graph $\rightarrow$ Verification
RelVeri+: SQL $\rightarrow$ Formal Verification

8.2 End-to-End Pipeline



8.3 Novel Contributions

Contribution 1: Predicate HyperEdges

VeriStrong models WR, WW and RW dependencies but does not explicitly represent predicate-based interactions.

RelVeri+ introduces Predicate HyperEdges (PH):

$$T_1 \xrightarrow{PH} T_2$$

where transaction T_2 modifies the result of a predicate query executed by T_1 .

This enables explicit reasoning about:

- Phantom Reads
- Predicate Conflicts
- Range Query Anomalies

Contribution 2: Automatic Hyper-Polygraph Generation

Current VeriStrong requires a pre-built Hyper-Polygraph.

RelVeri+ automatically constructs:

$$SQL \rightarrow \text{Dependency Graph} \rightarrow \text{Hyper-Polygraph}$$

using IsoRel's dependency extraction mechanism.

No manual graph construction is required.

Contribution 3: Anomaly Explanation Engine

Instead of reporting only SAT or UNSAT, RelVeri+ produces human-readable explanations.

Example:

- Anomaly: Write Skew
- Transactions: T_{12}, T_{17}
- Rows: DoctorA, DoctorB
- Cycle:

$$T_{12} \xrightarrow{\text{RW}} T_{17} \xrightarrow{\text{RW}} T_{12}$$

This greatly improves debugging and diagnosis.

8.4 Formal Algorithms

Algorithm 1: RelVeri+ Main Framework

Algorithm 1 RelVeri+ End-to-End Isolation Verification

Require: SQL Workload W

Ensure: Isolation Verdict

```

1: Execute workload  $W$ 
2: Instrument rows using rowId and wList
3: Collect execution history  $H$ 
4:  $G \leftarrow \text{BUILDDEPENDENCYGRAPH}(H)$ 
5:  $HP \leftarrow \text{BUILDHYPERPOLYGRAPH}(G)$ 
6:  $\text{ADDPREDICATEHYPEREDGES}(HP, H)$ 
7: if  $\text{DETECT1WIDTHCYCLE}(HP)$  then
8:   return IsolationViolation
9: end if
10:  $\text{APPLY2WIDTHPRUNING}(HP)$ 
11:  $\phi \leftarrow \text{ENCODETOSMT}(HP)$ 
12:  $\text{result} \leftarrow \text{Z3SOLVE}(\phi)$ 
13: if  $\text{result} = \text{SAT}$  then
14:   return Serializable
15: else
16:    $\text{EXPLAINVIOLATION}(HP)$ 
17:   return IsolationViolation
18: end if

```

Interpretation

This algorithm acts as the controller of the complete framework. It starts from a SQL workload, extracts transaction dependencies using IsoRel-inspired instrumentation, constructs a Hyper-Polygraph, performs cycle pruning, and finally invokes SMT verification. The output is either a proof that the execution is Serializable/Snapshot-Isolation compliant or a detailed explanation of the anomaly.

Algorithm 2: Build Hyper-Polygraph**Algorithm 2** Automatic Hyper-Polygraph Construction**Require:** Dependency Graph G **Ensure:** Hyper-Polygraph HP

- 1: Create a vertex for each committed transaction
- 2: Add all known WR edges
- 3: Add all known WW edges
- 4: Add all known RW edges
- 5: **for** each read with duplicate-value ambiguity **do**
- 6: Create WR Hyperedge Constraint
- 7: **end for**
- 8: **for** each uncertain version-order relation **do**
- 9: Create WW Hyperedge Constraint
- 10: **end for**
- 11: **return** HP

Interpretation

This algorithm automatically converts the dependency graph produced by IsoRel into a Hyper-Polygraph suitable for VeriStrong-style verification. Instead of selecting a single dependency immediately, all valid dependency choices are preserved as hyperedge constraints. This allows the SMT solver to explore all possible compatible graphs while remaining sound in the presence of duplicate values.

Algorithm 3: Predicate HyperEdge Generation**Algorithm 3** Predicate-Aware Dependency Construction**Require:** SQL History H , Hyper-Polygraph HP

- 1: **for** each SELECT query Q with predicate P **do**
- 2: **for** each concurrent INSERT or DELETE operation **do**
- 3: **if** affected row satisfies predicate P **then**
- 4: Add Predicate HyperEdge

$$T_Q \xrightarrow{PH} T_{op}$$

- 5: **end if**
- 6: **end for**
- 7: **end for**

Interpretation

This is the main extension proposed in RelVeri+. Existing Hyper-Polygraphs model WR, WW, and RW dependencies only. However, SQL workloads frequently contain predicate-based queries that may be affected by concurrent inserts or deletes. Predicate HyperEdges explicitly capture these interactions, enabling formal reasoning about Phantom Reads, range-query anomalies, and predicate conflicts.

8.5 Expected Capabilities

Table 3: Feature Comparison of IsoRel, VeriStrong and RelVeri+

Feature	IsoRel	VeriStrong	RelVeri+
SQL Support	✓	×	✓
Dependency Extraction	✓	×	✓
Hyper-Polygraph	×	✓	✓
SMT Verification	×	✓	✓
Duplicate Value Handling	Partial	✓	✓
Predicate Reasoning	Limited	×	✓
Phantom Detection	Limited	×	✓
End-to-End Verification	×	×	✓
Explanation Engine	×	×	✓

8.6 Research Contribution

The proposed RelVeri+ framework combines the strengths of IsoRel and VeriStrong into a single end-to-end isolation analysis pipeline. IsoRel contributes relational dependency extraction directly from SQL workloads, while VeriStrong contributes sound verification through Hyper-Polygraphs and SMT solving. RelVeri+ further extends these ideas through Predicate HyperEdges, automatic Hyper-Polygraph construction, and an anomaly explanation engine. As a result, the framework can detect, explain, and formally verify isolation anomalies in relational database systems using a unified workflow.

RelVeri+ bridges the gap between anomaly detection and formal verification by providing a complete SQL-to-SMT verification pipeline.

8.7 Prototype Implementation

A prototype implementation of RelVeri+ has been developed in Python using NetworkX for graph processing and Z3 for SMT verification.

The implementation is publicly available at:

<https://github.com/yadavvinodkumar6391-hue/DBMS-PROJECT-26>

9 Conclusion

This report studied two recent advances in database isolation analysis: **IsoRel** and **VeriStrong**.

IsoRel addresses the problem of isolation anomaly detection in relational database systems. By introducing relational-aware dependency inference through auxiliary metadata such

as `rowId` and `wList`, IsoRel can accurately construct dependency graphs and identify anomalies such as lost updates, read-write skew, and write skew in real SQL workloads.

VeriStrong addresses the complementary problem of isolation verification. Instead of detecting anomalies from a single dependency graph, it models all possible dependency graphs using the Hyper-Polygraph formalism and uses SMT solving to verify whether a transaction history satisfies strong isolation guarantees such as Snapshot Isolation and Serializability, even in the presence of duplicate values.

Together, these works provide a comprehensive foundation for database correctness analysis. IsoRel focuses on extracting relational dependencies from observed executions, while VeriStrong focuses on formally verifying the correctness of those dependencies.

Based on the strengths and limitations of both approaches, this report proposed **RelVeri+**, a unified SQL-to-SMT isolation verification framework that combines IsoRel’s relational dependency extraction with VeriStrong’s Hyper-Polygraph verification and extends both with predicate-aware anomaly reasoning and automated explanation generation. The proposed framework enables an end-to-end workflow from SQL workload execution to formal verification and anomaly diagnosis.

Future work may focus on supporting distributed database systems, real-time verification, cloud-native workloads, and machine-learning-based anomaly prediction to further improve the scalability and practicality of database isolation verification.

References

- [1] Rui Yang, Ziyu Cui, Wensheng Dou, Yu Gao, Jiansen Song, Xudong Xie, and Jun Wei. *Detecting Isolation Anomalies in Relational DBMSs*. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2025.
- [2] Zhiheng Cai, Si Liu, Hengfeng Wei, Yuxing Chen, and Anqun Pan. *Fast Verification of Strong Database Isolation*. In *Proceedings of the VLDB Endowment*, Volume 19, 2026.
- [3] Atul Adya. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*. PhD Thesis, Massachusetts Institute of Technology, 1999.
- [4] Hal Berenson, Philip Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. *A Critique of ANSI SQL Isolation Levels*. In *Proceedings of ACM SIGMOD*, 1995.
- [5] Kyle Kingsbury. *Elle: Inferring Isolation Anomalies from Experimental Observations*. In *Proceedings of the VLDB Endowment*, 2021.
- [6] Zuming Jiang, Jianyu Zhao, and Hengfeng Wei. *PolySI: Verifying Snapshot Isolation for Distributed Databases*. In *Proceedings of the VLDB Endowment*, 2023.